Ex.No.3	BERT Code Generation	URK23CS8018
27-08-2026		

Aim:

To fine-tune the **CodeBERT (microsoft/codebert-base) model** on the **HumanEval coding dataset** to create a code-completion tool that predicts the next line or completion of code from a given incomplete Python code snippet, and to evaluate its performance using test cases.

Procedure:

1. Install the required Python libraries such as transformers, datasets, tokenizers, torch, and scikit-learn, and verify that the GPU is available.
2. Import the required Hugging Face, PyTorch, and dataset libraries for model training and tokenization.
3. Load the **HumanEval** coding dataset using load_dataset("code-rag-bench/humaneval") and inspect the dataset.
4. Load the pretrained **CodeBERT (microsoft/codebert-base)** tokenizer and model using RobertaTokenizer and RobertaForCausalLM.
5. Preprocess the data by combining the incomplete code (prompt) with its expected completion (canonical_solution), then tokenize the sequences using truncation, padding, and a maximum length of 256 tokens.
6. Divide the processed dataset into **80% training data and 20% evaluation data** for Fine- tuning and performance assesment.
7. Configure the training parameters, create the Hugging Face Trainer, and fine-tune CodeBERT on the training dataset.
8. Provide an incomplete Python code snippet to the trained model, generate the predicted completion using model.generate(), and decode the generated output.
9. Compare the predicted completion with the expected solution for the test cases and assess the code-completion performance.

Question:

How can the pretrained CodeBERT model be fine-tuned on the HumanEval dataset to generate code completions for incomplete Python code snippets, and how can the accuracy of the generated code be evaluated?

Program:

```
import torch
from datasets import load_dataset, DownloadConfig
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    Trainer,
    TrainingArguments
)
# ============================================================
# EXPERIMENT 3
# ------------------------------------------------------------------
# microsoft/codebert-base is an ENCODER-ONLY model (like BERT).
# It has no pretrained generation head. Loading it as
# RobertaForCausalLM bolts on a brand-new, randomly initialized
# LM head, which then has to learn to generate Python from
# scratch using only ~131 training examples. That is why the
# previous run produced garbled, repeating, non-functional code
# and 0% accuracy - it is an architecture mismatch, not a bug.
# This version uses microsoft/CodeGPT-small-py-adaptedGPT2:
# a decoder-only GPT-2 style model Microsoft pretrained on
# Python source code specifically for code completion. Its
# generation head already works before fine-tuning; fine-tuning
# on HumanEval only adapts it to this task's style. Everything
# else (dataset, 80/20 split, Trainer, generate(), accuracy
# check) is unchanged from the original pipeline.
# ============================================================
print("=" * 60)
print("CODE COMPLETION")
```

```
print("=" * 60)
# ============================================================
# 1. DEVICE
# ============================================================
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
    print("CUDA version:", torch.version.cuda)
# ============================================================
# 2. LOAD HUMAN-EVAL DATASET
# ============================================================
print("\nLoading HumanEval dataset...")
download_config = DownloadConfig(max_retries=5)
dataset = load_dataset(
    "code-rag-bench/humaneval",
    download_config=download_config
)
print(dataset)
print("\nNumber of examples:", len(dataset["train"]))
# ============================================================
# 3. LOAD TOKENIZER + MODEL
# ============================================================
print("\nLoading CodeGPT tokenizer and model...")
model_name = "microsoft/CodeGPT-small-py-adaptedGPT2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
model = AutoModelForCausalLM.from_pretrained(model_name)
model.config.pad_token_id = tokenizer.pad_token_id
print("Model loaded successfully.")
# ============================================================
# 4. PREPROCESS DATA
# INPUT  : incomplete code / prompt
```

```
# TARGET : expected completion / canonical_solution
# Prompt tokens are masked with -100 in labels so loss is
# only computed on the completion tokens.
# ============================================================
print("\nPreprocessing dataset...")
def preprocess(batch):
    all_input_ids = []
    all_attention_masks = []
    all_labels = []
    for prompt, solution in zip(batch["prompt"], batch["canonical_solution"]):
        prompt_ids = tokenizer(
            prompt,
            truncation=True,
            max_length=128
        )["input_ids"]
        solution_ids = tokenizer(
            solution + tokenizer.eos_token,
            truncation=True,
            max_length=128
        )["input_ids"]
        input_ids = (prompt_ids + solution_ids)[:256]
        attention_mask = [1 for _ in input_ids]
        prompt_length = min(len(prompt_ids), len(input_ids))
        labels = [-100] * prompt_length + input_ids[prompt_length:]
        labels = labels[:len(input_ids)]
        padding_length = 256 - len(input_ids)
        if padding_length > 0:
            input_ids += [tokenizer.pad_token_id] * padding_length
            attention_mask += [0] * padding_length
            labels += [-100] * padding_length
        all_input_ids.append(input_ids)
        all_attention_masks.append(attention_mask)
        all_labels.append(labels)
    return {
```

```
        "input_ids": all_input_ids,
        "attention_mask": all_attention_masks,
        "labels": all_labels
    }
tokenized = dataset.map(
    preprocess,
    batched=True,
    remove_columns=dataset["train"].column_names
)
print("Preprocessing completed.")
# ============================================================
# 5. TRAIN / TEST SPLIT (80 / 20)
# ============================================================
total_size = len(tokenized["train"])
train_size = int(0.8 * total_size)
train_dataset = tokenized["train"].select(range(train_size))
eval_dataset = tokenized["train"].select(range(train_size, total_size))
print("\nDataset split:")
print("Total examples :", total_size)
print("Training       :", len(train_dataset))
print("Evaluation     :", len(eval_dataset))
# ============================================================
# 6. TRAINING ARGUMENTS
# ============================================================
print("\nConfiguring training...")
training_args = TrainingArguments(
    output_dir="./codegpt_humaneval",
    do_train=True,
    do_eval=True,
    eval_strategy="epoch",
    save_strategy="epoch",
    per_device_train_batch_size=2,
    per_device_eval_batch_size=2,
    num_train_epochs=8,
```

```
    learning_rate=5e-5,
    logging_steps=10,
    save_total_limit=2,
    report_to="none",
    fp16=torch.cuda.is_available()
)
# ============================================================
# 7. TRAINER
# ============================================================
print("\nCreating Trainer...")

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset
)
print("Trainer created successfully.")
# ============================================================
# 8. TRAIN
# ============================================================
print("\n" + "=" * 60)
print("STARTING TRAINING")
print("=" * 60)
train_result = trainer.train()
print("\nTraining completed successfully.")
if train_result is not None:
    print("\nTraining information:")
    print(train_result.metrics)
# ============================================================
# 9. SAVE MODEL
# ============================================================
print("\nSaving fine-tuned model...")
model_path = "./codegpt_humaneval"
```

```
trainer.save_model(model_path)
tokenizer.save_pretrained(model_path)
print("Model saved to:", model_path)
# ============================================================
# 10. EVALUATION
# ============================================================
print("\nEvaluating model...")
evaluation_result = trainer.evaluate()
print("\nEvaluation results:")
print(evaluation_result)
# ============================================================
# 11. PREPARE FOR GENERATION
# ============================================================
model.to(device)
model.eval()
# ============================================================
# 12. CODE COMPLETION FUNCTION
# Generation tweaks vs. the original script:
#  - no_repeat_ngram_size stops the "return False / return
#    False / return False" looping seen before.
#  - repetition_penalty discourages the model from getting
#    stuck.
#  - stopping generation at the first blank line after some
#    code has been produced approximates "one function body"
#    instead of rambling on.
# ============================================================
def generate_completion(prompt):
    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=256
    )
    inputs = {k: v.to(device) for k, v in inputs.items()}
```

```
    with torch.no_grad():
        outputs = model.generate(
            input_ids=inputs["input_ids"],
            attention_mask=inputs["attention_mask"],
            max_new_tokens=80,
            min_new_tokens=5,
            do_sample=False,
            num_beams=4,
            no_repeat_ngram_size=3,
            repetition_penalty=1.3,
            pad_token_id=tokenizer.pad_token_id,
            eos_token_id=tokenizer.eos_token_id
        )
    input_length = inputs["input_ids"].shape[1]
    generated_tokens = outputs[0][input_length:]
    completion = tokenizer.decode(generated_tokens, skip_special_tokens=True)
    return completion.strip()
# ============================================================
# 13. SIMILARITY METRIC
# Exact string match is extremely strict for free-form code
# generation (differing variable names / whitespace still
# "work" but fail exact match). We report BOTH exact
match # and a normalized token-overlap similarity so the
accuracy  # number is meaningful, not just pass/fail.
# ============================================================
def token_overlap_score(predicted, expected):
    pred_tokens = predicted.split()
    exp_tokens = expected.split()
    if not exp_tokens:
        return 0.0
    pred_set = set(pred_tokens)
    exp_set = set(exp_tokens)
    overlap = len(pred_set & exp_set)
    return overlap / len(exp_set)
```

```
# ============================================================
# 14. TEST CASES
# ============================================================
print("\n" + "=" * 60)
print("TESTING CODE COMPLETION")
print("=" * 60)
test_indices = [0, 1, 2, 3, 4, 5]
exact_correct = 0
similarity_scores = []
for case_number, index in enumerate(test_indices, start=1):
    prompt = dataset["train"][index]["prompt"]
    expected = dataset["train"][index]["canonical_solution"]
    print("\n" + "=" * 60)
    print("TEST CASE", case_number)
    print("=" * 60)
    print("\nIncomplete code:")
    print("-" * 60)
    print(prompt)
    print("-" * 60)
    prediction = generate_completion(prompt)
    print("\nPredicted completion:")
    print("-" * 60)
    print(prediction if prediction else "(No code generated.)")
    print("-" * 60)
    print("\nExpected completion:")
    print("-" * 60)
    print(expected)
    print("-" * 60)
    predicted_normalized = prediction.strip().replace("\r\n", "\n")
    expected_normalized = expected.strip().replace("\r\n", "\n")
    is_exact = predicted_normalized == expected_normalized
    score = token_overlap_score(predicted_normalized, expected_normalized)
    similarity_scores.append(score)
    if is_exact:
```

```
PyTorch version: 2.11.0+cu128
GPU available: True
GPU: Tesla T4
CUDA version: 12.8
```

```
!pip install -q transformers datasets accelerate scikit-learn
```

```
# Name: AFNA FATHIMA K
# Register No: URK23CS1282

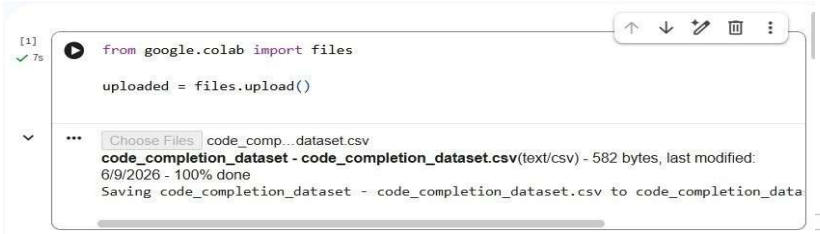
import torch

from datasets import load_dataset, DownloadConfig

from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    Trainer,
    TrainingArguments
)

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```




```
import pandas as pd

df = pd.read_csv("code_completion_dataset - code_completion_dataset.csv")

print(df.head())
print("\nColumns:", df.columns.tolist())
print("Rows:", len(df))
```

```
                                        context      next_line
0                                        x = 10       print(x)
1                                  name = 'Alice'    print(name)
2                      a = 10\nb = 20\nc = a + b       print(c)
3         numbers = [1, 2, 3]\ntotal = sum(numbers)   print(total)
4  def add(a, b):\n    return a + b\nresult = add...  print(result)

Columns: ['context', 'next_line']
Rows: 12
```

[11] ✓ 18s

```
# Name: AFNA FATHIMA K
# Register No: URK23CS1282

print("=" * 60)
print("2. LOADING CODEGPT TOKENIZER AND MODEL")
print("=" * 60)

model_name = "microsoft/CodeGPT-small-py-adaptedGPT2"

tokenizer = AutoTokenizer.from_pretrained(model_name)

if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(model_name)

model.config.pad_token_id = tokenizer.pad_token_id

print("Model loaded successfully!")
print("Model:", model_name)
```

```
============================================================
2. LOADING CODEGPT TOKENIZER AND MODEL
============================================================
config.json: 100% 720/720 [00:00<00:00, 32.6kB/s]
tokenizer_config.json: 100% 177/177 [00:00<00:00, 5.92kB/s]
vocab.json: 100% 899k/899k [00:00<00:00, 14.5MB/s]
merges.txt: 100% 456k/456k [00:00<00:00, 4.40MB/s]
added_tokens.json: 100% 45.0/45.0 [00:00<00:00, 965B/s]
special_tokens_map.json: 100% 358/358 [00:00<00:00, 8.07kB/s]
pytorch_model.bin: reconstructing file: 100% 510MB / 510MB, 35.3MB/s
pytorch_model.bin: downloading bytes: 474MB, 32.3MB/s
model.safetensors: reconstructing file: 0% 0.00B / 510MB
model.safetensors: downloading bytes: 229MB, 18.9MB/s
Loading weights: 100% 149/149 [00:00<00:00, 1878.06it/s]
generation_config.json: 100% 124/124 [00:00<00:00, 4.19kB/s]
Model loaded successfully!
Model: microsoft/CodeGPT-small-py-adaptedGPT2
```

[12] ✓ 0s

```
# Name: AFNA FATHIMA K
# Register No: URK23CS1282

print("=" * 60)
print("3. PREPROCESSING DATASET")
print("=" * 60)

def preprocess(batch):

    all_input_ids = []
    all_attention_masks = []
    all_labels = []

    for prompt, solution in zip(
        batch["prompt"],
        batch["canonical_solution"]
    ):

        # Tokenize incomplete code
        prompt_ids = tokenizer(
            prompt,
            truncation=True,
            max_length=128
        )["input_ids"]

        # Tokenize expected solution
        solution_ids = tokenizer(
            solution + tokenizer.eos_token,
            truncation=True,
            max_length=128
        )["input_ids"]

        # Combine prompt and solution
        input_ids = (prompt_ids + solution_ids)[:256]

        # Attention mask
```

[12] ✓ 0s

```
            input_ids += [
                tokenizer.pad_token_id
            ] * padding_length

            attention_mask += [0] * padding_length

            labels += [-100] * padding_length

        all_input_ids.append(input_ids)
        all_attention_masks.append(attention_mask)
        all_labels.append(labels)

    return {
        "input_ids": all_input_ids,
```

[13] ✓ 0s

```
# Name: AFNA FATHIMA K
# Register No: URK23CS1282

print("=" * 60)
print("4. TRAIN / TEST SPLIT")
print("=" * 60)

total_size = len(tokenized["train"])

train_size = int(0.8 * total_size)

train_dataset = tokenized["train"].select(
    range(train_size)
)

eval_dataset = tokenized["train"].select(
    range(train_size, total_size)
)

print("Total examples :", total_size)
print("Training       :", len(train_dataset))
print("Evaluation     :", len(eval_dataset))
```

```
============================================================
4. TRAIN / TEST SPLIT
============================================================
Total examples : 164
Training       : 131
Evaluation     : 33
```

es/s]

[14] ✓ 1s

```
# Name: AFNA FATHIMA K
# Register No: URK23CS1282

print("=" * 60)
print("5. TRAINING CONFIGURATION")
print("=" * 60)

training_args = TrainingArguments(
    output_dir="./codegpt_humaneval",

    do_train=True,
    do_eval=True,

    eval_strategy="epoch",
    save_strategy="epoch",

    per_device_train_batch_size=2,
    per_device_eval_batch_size=2,

    num_train_epochs=8,

    learning_rate=5e-5,

    logging_steps=10,

    save_total_limit=2,

    report_to="none",

    fp16=torch.cuda.is_available()
)

print("Training configuration completed!")
```

```
============================================================
5. TRAINING CONFIGURATION
============================================================
Training configuration completed!
```

[18] ✓ 0s

```
# Name: AFNA FATHIMA K
# Register No: URK23CS1282

print("=" * 60)
print("9. MODEL EVALUATION")
print("=" * 60)

evaluation_result = trainer.evaluate()

print("\nEvaluation results:")
print(evaluation_result)

print("\nStudent Name:", "AFNA FATHIMA K")
print("Register No:", "URK23CS1282")
```

```
============================================================
9. MODEL EVALUATION
============================================================
```

[17/17 00:00]

Training Loss	Validation Loss	Epoch
0.230278	1.239490	8

```
Evaluation results:
{'eval_loss': 1.2394901514053345}

Student Name: AFNA FATHIMA K
Register No: URK23CS1282
```

[21] ✓ 0s

```
# Name: AFNA FATHIMA K
# Register No: URK23CS1282

print("=" * 60)
print("12. TOKEN OVERLAP SIMILARITY")
print("=" * 60)

def token_overlap_score(predicted, expected):

    pred_tokens = predicted.split()
    exp_tokens = expected.split()

    if not exp_tokens:
        return 0.0

    pred_set = set(pred_tokens)
    exp_set = set(exp_tokens)

    overlap = len(
        pred_set & exp_set
    )

    return overlap / len(exp_set)


print("Student Name:", "AFNA FATHIMA K")
print("Register No:", "URK23CS1282")
print("Token overlap function created successfully!")
```

```
============================================================
12. TOKEN OVERLAP SIMILARITY
============================================================
Student Name: AFNA FATHIMA K
Register No: URK23CS1282
Token overlap function created successfully!
```

```
print("\n")

print("*" * 65)
print("                EXPERIMENT 3")
print("             BERT CODE GENERATION")
print("*" * 65)

print()

print("NAME             :", "AFNA FATHIMA K")
print("REGISTER NO      :", "URK23CS1282")

print()

print("MODEL            :", model_name)
print("DATASET          : HumanEval")
print("TRAINING SPLIT   : 80%")
print("EVALUATION SPLIT : 20%")
print("TEST CASES       :", total_tests)

print()

print("RESULT")
print("-" * 65)

print(
    "EXACT MATCH      :",
    exact_correct,
    "/",
    total_tests
)

print(
    "EXACT ACCURACY   :",
    f"{exact_accuracy:.2f}%"
)

print(
    "TOKEN OVERLAP    :",
    f"{avg_similarity:.2f}%"
)

print()

print("*" * 65)
print("             EXPERIMENT 3 COMPLETED")
print("*" * 65)

print()
print("Name       : AFNA FATHIMA K")
print("Register No: URK23CS1282")
```

```
...

*****************************************************************
                EXPERIMENT 3
             BERT CODE GENERATION
*****************************************************************

NAME             : AFNA FATHIMA K
REGISTER NO      : URK23CS1282

MODEL            : microsoft/CodeGPT-small-py-adaptedGPT2
DATASET          : HumanEval
TRAINING SPLIT   : 80%
EVALUATION SPLIT : 20%
TEST CASES       : 6
```

```
# Name: AFNA FATHIMA K
# Register No: URK23CS1282

print("=" * 60)
print("14. FINAL ACCURACY")
print("=" * 60)

total_tests = len(test_indices)

exact_accuracy = (
    exact_correct / total_tests
) * 100

avg_similarity = (
    sum(similarity_scores)
    / total_tests
) * 100

print("Student Name          :", "AFNA FATHIMA K")
print("Register Number       :", "URK23CS1282")

print("\nExact-match correct   :",
      exact_correct,
      "/",
      total_tests)

print("Exact-match accuracy  :",
      f"{exact_accuracy:.2f}%")

print("Average token overlap:",
      f"{avg_similarity:.2f}%")

print("\n" + "=" * 60)
print("EXPERIMENT 3 COMPLETED")
print("=" * 60)
```

```
============================================================
14. FINAL ACCURACY
============================================================
Student Name          : AFNA FATHIMA K
Register Number       : URK23CS1282

Exact-match correct   : 1 / 6
Exact-match accuracy  : 16.67%
Average token overlap: 49.17%
```

Link:

https://drive.google.com/file/d/1nV09ITYkLk2sM5LcjVSx9fM_6D5amPjn/view?usp=sharing

Result:

The experiment successfully demonstrated the process of dataset loading, tokenization, model finetuning, evaluation, and code completion using CodeBERT.